

Publish In Maven Central Repository

Maven-source-plugin

- The Maven Source Plugin is a tool that provides the ability to attach the source code of a project to the project's main artifact.
- This can be very useful for developers who want to debug a library or simply explore the source code.
- It can be configured to include/exclude specific files and directories.

Maven-javadoc-plugin

- The Maven Javadoc Plugin is a tool used to generate Javadoc documentation for your Java project.
- This plugin can be configured to create HTML documentation from your Java source files.
- Configure the appearance and content of the Javadocs and attach the generated Javadocs to the project's main artifact.

Maven-gpg-plugin

- The Maven GPG Plugin is used to sign your project's artifacts using GNU Privacy Guard (GPG). Signing artifacts is crucial for verifying the integrity and authenticity of the artifacts you publish.
- This plugin can be configured to sign your artifacts during the deployment phase.

Need :

GPG signing ensures that the artifact has not been altered or corrupted since it was signed. If any changes are made to the artifact after it has been signed, the signature verification will fail, alerting users to the issue.

Signing an artifact with your GPG key proves that it was indeed created by you. This helps users to trust the source of the artifact. When they verify the signature, they can be confident that the artifact comes from a trusted and verified source.

Central-publishing-maven-plugin

- The Central Repository Publishing Maven Plugin simplifies the process of publishing artifacts to Maven Central, one of the most popular repositories for Java libraries.
- This plugin helps automate the steps required to deploy your artifacts, including signing them with GPG, creating the necessary POM files, and uploading the artifacts to the Central Repository.

Generating GPG keys simply can be done with this script and manually as your process.

```
#!/bin/bash
# Prompt for user input
read -p "Enter your full name: " NAME
read -p "Enter your email address: " EMAIL
read -p "Enter your Passphrase: " PASS
# Generate the GPG key
gpg --batch --generate-key <<EOF
Key-Type: RSA
Key-Length: 4096
Name-Real: $NAME
Name-Email: $EMAIL
Expire-Date: 0
Passphrase: $PASS
%commit
EOF
echo "GPG key generated successfully."
```

Configuration of maven server and GPG server done in the part of **settings.xml**

A screenshot of a code editor showing the configuration for Maven servers in a settings.xml file. The XML structure is as follows: <settings> <servers> <server> <id>{server}</id> <username>{username}</username> <password>{password}</password> </server> <server> <id>{id}</id> <passphrase>{passphrase}</passphrase> </server> </servers> </settings>. The code is color-coded: <settings> is red, <servers> is blue, <server> is blue, <id> is green, {server} and {id} are black, </id> is green, <username> is blue, {username} is black, </username> is blue, <password> is blue, {password} is black, </password> is blue, <passphrase> is blue, {passphrase} is black, </passphrase> is blue, and </server> and </settings> are blue.

```
<settings>
  <servers>
    <server>
      <id>{server}</id>
      <username>{username}</username>
      <password>{password}</password>
    </server>
    <server>
      <id>{id}</id>
      <passphrase>{passphrase}</passphrase>
    </server>
  </servers>
</settings>
```

The configuration and process can be referred to in [publishing by using the Maven Plugin](#).